cocos开发文档

Cocos开发文档

1.0 功能解决方案

1.0.1 底部对齐(使用widget偏移量)

用 （屏幕高度-安全区高度）/ 2 得到底部可显示区域的最小偏移量

```
let bpttomHeigh = (wx.getSystemInfoSync().screenHeight - wx.getSystemInfoSync().safeArea.height) / 2;
```

将得到的值做完y轴的值转换成世界vec3坐标

```
let outPos:Vec3=this.cam.getComponent(Camera).screenToWorld(new Vec3(0,bpttomHeigh,0));
```

将物体对齐高度设置成转换后坐标的y轴值

```
this.mainPanel.getChildByPath("BG/Foreground").getComponent(Widget).bottom=outPos.y;
```

若要将物体移至屏幕最下端贴合则直接使用0就行了

```
let outPos: Vec3 = cam.getComponent(Camera).screenToWorld(new Vec3(0, 0, 0));
```

1.0.2 顶部对齐(使用widget偏移量)

用 （屏幕高度-安全区高度）得到顶部可显示区域的偏移量,**注意不要/2，因为有刘海会遮挡**

```
let bpttomHeigh=(wx.getSystemInfoSync().screenHeight-wx.getSystemInfoSync().safeArea.height);
```

同1.0.1一样转换成世界坐标后，设置顶部对齐高度

```
this.mainPanel.getChildByPath("UiLayer/TopArea").getComponent(Widget).top=outPos.y;
```

以上1.0.1、1.0.2方法部分安卓机不适用，可采用1.0.3

1.0.3 使用SafeArea组件

创建一个空节点，挂载Widget上下左右拉伸对齐铺满画布，再挂载safearea组件，safearea组件对其子节点自动生效，注意子节点也需挂载Widget，根据需求选择对齐方向

1.1 动态滚动列表加载

1.1.1 向下加载

判断（【当前列表顺序】 + 【物体数量】）是否小于 【列表长度】同时判断 当前滑动列表y轴坐标有没有超出 （【当前显示列表起始物体y轴】+ 【页面里物体数量】 * 【物体高度】）

```
if(this.pointer+24 < 100 && this.listcontent.position.y>=this.listStartY+16*120)
```

如果为true 首先等待滚动结束

```
if(this.board.getChildByPath("RankView").getComponent(ScrollView).isAutoScrolling())
{
    return;
}
```

然后 让【当前列表顺序】后移 n 个

```
let downLoad=8;
this.pointer+=downLoad;
```

判断后移的位置是否超出列表上限 如果超出则回退至列表范围内

```
if(this.pointer+24>100)//100记得改成字典的长度
{
    let outLen=this.pointer+24-100;//100记得改成字典的长度
    downLoad-=outLen;
    this.pointer-=downLoad;
}
```

此间执行更新物体代码

将滑动列表的y轴向下移 【后移n位】 * 【物体高度】

```
this.listcontent.setPosition(new Vec3(0,this.listcontent.position.y-downLoad*120,0));
```

1.1.2 向上加载

同 1.1.1 将代码反过来即可

```
if(this.pointer>0 && this.listcontent.position.y<= this.listStartY)
{
    if(this.board.getChildByPath("RankView").getComponent(ScrollView).isAutoScrolling())
    {
        return;
    }
    let upLoad=8;
    this.pointer-=upLoad;
    if(this.pointer<0)
    {
        upLoad+=this.pointer;
        this.pointer=0;
    }
    //此处更新物体信息
    //.....
    this.listcontent.setPosition(new Vec3(0,this.listcontent.position.y+upLoad*120,0));
}
```

2.0 常见bug

2.0.1 spine骨骼动画加载错误

如果是从Bundle中读取骨骼动画，不能使用异步的方法读取bundle，但为了不阻塞程序运行可以用回调的方式传递对象，代码如下：

```
loadAssetsFromBundleSync(type: __private._types_globals__Constructor<Asset> ,bundleRes:string, assetsRes:string ,_callBack:(data)=>void)
{
    try
    {
        let bundle : AssetManager.Bundle = null;
        if(this.bundles.has(bundleRes))
        {
            bundle = this.bundles.get(bundleRes);
            bundle.load(assetsRes, type, (error, data) =>
            {
                if (error)
                {
                    console.warn("loadAssetsFromBundleSync 读取资源失败 :  ", error.message);
                    _callBack(null);
                }
                else
                {
                    _callBack(data);
                }
            });
        }
        else
        {
            assetManager.loadBundle(bundleRes,(error,bundle)=>
                {
                    if(error)
                    {
                        console.warn("loadAssetsFromBundleSync 读取bundle失败 :  ", error.message);
                        _callBack(null);
                    }
                    else
                    {
                        this.bundles.set(bundleRes, bundle);
                        bundle.load(assetsRes, type ,(error,data)=>
                        {
                            if(error)
                            {
                                console.warn("loadAssetsFromBundleSync 读取资源失败 :  ", error.message);
                                _callBack(null);
                            }
                            else
                            {
                                _callBack(data);
                            }
                        });
```

再用一个异步函数调用即可，代码如下：

```
public static LoadSkeletonData(_address:string,_callBack:(data)=>void):Promise<void>
{
    return new Promise(async(resolve)=>
    {
        try
        {
            let ads: string[] = null;
            if (_address)
            {
                ads = _address.split('/');
                BundleManager.Instance.loadAssetsFromBundleSync(sp.SkeletonData ,ads[0], `${ads[1]}/${ads[2]}`,(data)=>
                {
                    if(data)
                    {
                        _callBack(data);
                    }
                    else
                    {
                        console.warn(`loadAssets 里的 LoadSkeletonData 异常 : 路径${_address}下没有相对应资源,替换为默认`);
                        //temp=await BundleManager.Instance.loadAssetsFromBundle("RoleSpine", "Role_100004/kuangfeng_moshushi.skel") as sp.SkeletonData;
                        _callBack(null);
                    }
                });
            }
        }
        catch (error)
        {
            console.error('loadAssets 下 LoadSkeletonData 错误 err: ', error);
        }
        resolve();
    });
}
```